

**AUTOMATED PRESENTATION GENERATION FROM VIDEO
USING AI**

By

RAHUL SINGH A

Roll No. 2434MCA0047 Reg. No. 67224100047

A PROJECT REPORT

Submitted to the

FACULTY OF INFORMATION AND COMMUNICATION ENGINEERING

in partial fulfillment for the award of the degree

Of

MASTER OF COMPUTER APPLICATIONS



CENTRE FOR DISTANCE EDUCATION

ANNA UNIVERSITY

CHENNAI 600 025

December 2025

BONAFIDE CERTIFICATE

Certified that the project report titled "**Automated Presentation Generation from Video Using AI**" is the Bonafide work of Mr. **RAHUL SINGH A** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Date : _____

Signature of Student

Mr. RAHUL SINGH A

Roll No: 2434MCA0047

Register NO: 67224100047

Signature of Guide

Dr. P. Geetha,

Associate Professor,

Department of Information Science and
Technology,

College of Engineering, Guindy,

Anna University, Chennai - 25

CERTIFICATE OF VIVA – VOCE EXAMINATION

This is to certify that Mr. **RAHUL SINGH A** (Roll No: 2434MCA0047, Register No: 67224100047) has been subjected to Viva – voce Examination on _____ at the Study Centre: Centre for Distance Education, Anna University.

Internal Examiner

External Examiner

Name :

Name :

Designation :

Designation :

Address :

Address :

COORDINATOR STUDY CENTER

Name :

Designation :

Address :

ACKNOWLEDGEMENT

I extend my sincere gratitude to the faculty at Anna University, Chennai for their invaluable support and guidance throughout this project. I would like to express my deep appreciation to my course instructor at the Centre for Distance Education, Anna University, Chennai – 25, whose encouragement and insightful feedback were instrumental throughout the course of this work.

I am especially grateful to my project guide for their continuous support and highly valuable technical suggestions. Their expertise in Artificial Intelligence, Natural Language Processing, and Web Technologies significantly enriched the quality and depth of this project. Every discussion and review session proved to be immensely helpful in shaping the system architecture and implementation strategy.

I also extend my sincere thanks to the Anna University, Centre for Distance Education, for providing a stimulating academic environment and access to resources that made this research possible. The knowledge and skills acquired through this program have been invaluable.

My heartfelt thanks go to my classmates and peers who shared ideas, participated in testing sessions, and provided constructive feedback that helped improve the system iteratively. Their enthusiasm for technology and collaborative spirit made this journey enjoyable and productive.

Finally, I want to express my deepest gratitude to my parents and family members for their constant support, patience, and encouragement throughout this journey. They have been my backbone on any given day, and this achievement is as much theirs as it is mine.

(RAHUL SINGH A)

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	INTRODUCTION	15
	1.1 Overview	15
	1.2 Objectives	17
	1.3 Problem Statement	18
	1.4 Literature Survey	19
	1.5 Existing System	22
	1.6 Need for the System	23
	1.7 Scope of the Project	24
2	REQUIREMENTS SPECIFICATION	27
	2.1 Overview Description	27
	2.2 Key Features	27
	2.3 Hardware and Software Requirements	29
	2.4 Use Case Diagram	31
	2.5 Data Flow Diagram	32
3	SYSTEM DESIGN	35
	3.1 Proposed System	35
	3.2 System Architecture	36
	3.3 Repository Structure	37
	3.4 API Design	39
	3.5 Database Schema	40
	3.6 List of Modules	41
	3.7 Overall System Design	42
	3.8 System Testing	43

Chapter No.	Title	Page No.
	3.9 Test Cases	45
4	IMPLEMENTATION AND ANALYSIS OF RESULTS	49
	4.1 Implementation Environment	49
	4.2 Technology Stack	50
	4.3 Modules Completed	51
	4.4 Deviations and Justifications	56
	4.5 Project Roadmap and Status	58
5	CONCLUSION AND FUTURE WORK	60
	5.1 Limitations	60
	5.2 Future Enhancement	61
	5.3 Conclusion	63
	References	66
	Appendices	68

LIST OF TABLES

Table No.	Title of Table	Page No.
2.1	Hardware Requirements	
2.2	Software Requirements	
3.1	API Endpoint Summary	
3.2	Database Entity Relationships	
3.3	List of Modules	
3.4	Test Cases – Authentication Module	
3.5	Test Cases – Video Input Processing Module	
3.6	Test Cases – Audio Transcription Module	
3.7	Test Cases – NLP Content Structuring Module	
3.8	Test Cases – PPTX Generation and Export	
4.1	Technology Stack	
4.2	Modules Completed Summary	
4.3	Deviations and Justifications	
4.4	Project Roadmap and Phase Status	

LIST OF FIGURES

Figure No.	Title of Figure	Page No.
2.1	Use Case Diagram	
2.2	Data Flow Diagram – Level 0	
2.3	Data Flow Diagram – Level 1	
3.1	System Architecture Diagram	
3.2	Project Repository Structure	
3.3	Entity Relationship (ER) Diagram	
3.4	Prisma Database Schema	
4.1	User Authentication Flow	
4.2	Video Upload and Processing Interface	
4.3	AssemblyAI Transcription Pipeline	
4.4	LSA and YAKE NLP Processing Flow	
4.5	Gemini AI Slide Enhancement Output	
4.6	PPTX Generation Module Output	
4.7	User Dashboard Interface	
4.8	Slide Editor Interface	

ABSTRACT

In today's digital era, a vast amount of knowledge is shared through video content such as lectures, webinars, and training sessions. However, converting this unstructured multimedia content into well-organized presentations is time-consuming and requires significant effort.

This project, titled "**Automated Presentation Generation from Video Using Artificial Intelligence**" introduces an intelligent web-based system that automatically converts video and audio into structured PowerPoint (.pptx) presentations. Users can provide input via YouTube URLs or upload local media files, which are securely processed and transcribed using AssemblyAI for accurate speech-to-text conversion.

The transcript is then analyzed using Natural Language Processing techniques, combining Latent Semantic Analysis (LSA) for topic segmentation and YAKE for keyword extraction. This ensures the content is divided into meaningful slide sections with appropriate titles and bullet points.

Further refinement is done using Google Gemini 2.5, which enhances formatting, layouts, and visual design. The generated HTML-based slides are converted into PowerPoint files using a Python FastAPI backend. The system is built with a modern microservice architecture using Next.js, Express.js, FastAPI, and PostgreSQL with Prisma.

Overall, the platform provides a complete automated pipeline—from video input to final presentation—while allowing users to view and edit slides through a dashboard. This significantly reduces manual effort and demonstrates the power of AI-driven content transformation for educational and professional use.

சுருக்கம்

இன்றைய டிஜிட்டல் காலத்தில், விரிவான அளவில் அறிவு வீடியோ உள்ளடக்கங்கள் மூலம் பகிரப்படுகிறது; இதில் ஆன்லைன் வகுப்புகள், வெபினார்கள், மற்றும் பயிற்சி அமர்வுகள் அடங்கும். ஆனால், இந்த அமைப்பில்லாத மல்டிமீடியா உள்ளடக்கத்தை ஒழுங்குபடுத்தப்பட்ட பிரெசண்டேஷன்களாக மாற்றுவது நேரம் எடுத்துக்கொள்ளும் மற்றும் அதிக முயற்சி தேவைப்படும் செயலாகும்.

"Automated Presentation Generation from Video Using Artificial Intelligence" என்ற இந்த திட்டம், வீடியோ மற்றும் ஆடியோ உள்ளடக்கங்களை தானாகவே கட்டமைக்கப்பட்ட PowerPoint (.pptx) பிரெசண்டேஷன்களாக மாற்றும் ஒரு புத்திசாலி வலை அடிப்படையிலான அமைப்பை அறிமுகப்படுத்துகிறது. பயனர்கள் YouTube இணைப்புகள் அல்லது உள்ளூர் மீடியா கோப்புகளை பதிவேற்றம் செய்வதன் மூலம் உள்ளீட்டை வழங்கலாம்; அவை பாதுகாப்பாக செயலாக்கப்பட்டு AssemblyAI மூலம் துல்லியமான உரை மாற்றமாக (speech-to-text) மாற்றப்படுகின்றன.

அதன்பின், உரை Natural Language Processing (NLP) தொழில்நுட்பங்களைப் பயன்படுத்தி பகுப்பாய்வு செய்யப்படுகிறது; இதில் Latent Semantic Analysis (LSA) மூலம் தலைப்பு பிரிப்பும், YAKE மூலம் முக்கிய சொற்கள் எடுப்பும் செய்யப்படுகிறது. இதனால் உள்ளடக்கம் பொருத்தமான தலைப்புகள் மற்றும் முக்கிய குறிப்புகளுடன் ஸ்லைட்களாக பிரிக்கப்படுகிறது.

மேலும், Google Gemini 2.5 பயன்படுத்தி வடிவமைப்பு, அமைப்பு மற்றும் காட்சியமைப்பு மேம்படுத்தப்படுகிறது. உருவாக்கப்பட்ட HTML அடிப்படையிலான ஸ்லைட்கள் Python FastAPI backend மூலம் PowerPoint கோப்புகளாக மாற்றப்படுகின்றன. இந்த அமைப்பு Next.js, Express.js, FastAPI மற்றும் PostgreSQL (Prisma ORM உடன்) ஆகியவற்றைக் கொண்டு உருவாக்கப்பட்ட நவீன மைக்ரோசர்வீஸ் கட்டமைப்பில் இயங்குகிறது.

மொத்தத்தில், இந்த தளம் வீடியோ உள்ளீட்டிலிருந்து இறுதி பிரெசண்டேஷன் வரை முழுமையான தானியங்கி செயல்முறையை வழங்குகிறது; மேலும் பயனர்கள் தங்களது ஸ்லைட்களை பார்வையிடவும் திருத்தவும் ஒரு டாஷ்போர்டு வழங்கப்படுகிறது. இது கையேடு முயற்சியை கணிசமாக குறைத்து, கல்வி மற்றும் தொழில்முறை பயன்பாடுகளுக்கான AI அடிப்படையிலான உள்ளடக்க மாற்றத்தின் திறனை வெளிப்படுத்துகிறது.

CHAPTER I

INTRODUCTION

1.1 OVERVIEW

In the contemporary digital landscape, the proliferation of video-based educational content has created an unprecedented challenge for knowledge management and accessibility. Platforms such as YouTube and institutional learning management systems host billions of hours of lecture recordings, tutorial videos, webinars, and corporate training content. While this wealth of information is invaluable, its unstructured nature makes it difficult to review, summarize, and reference efficiently.

Presentation slides represent one of the most widely used formats for structured knowledge communication in academic and professional settings. They provide a concise, visual, and logically organized representation of information that facilitates learning, retention, and communication. However, creating high-quality presentation slides from video content is a labour-intensive process that requires multiple manual steps: watching the entire video, identifying key concepts, extracting relevant information, structuring it logically, and designing visually appealing slides.

The advent of Artificial Intelligence, particularly advances in Natural Language Processing (NLP), Large Language Models (LLMs), and cloud-based AI services, has opened transformative possibilities for automating this content conversion process. Automatic Speech Recognition (ASR) systems can now transcribe spoken language with near-human accuracy. Semantic analysis algorithms can identify topics and extract keywords from transcripts. Generative AI models can structure and enhance content presentation with professional-grade design.

This project, titled "Automated Presentation Generation from Video Using Artificial Intelligence," aims to leverage these cutting-edge technologies to build a comprehensive, end-to-end system that automatically converts video and audio content into professionally formatted PowerPoint presentations. The system accepts three types of input: YouTube video URLs, locally uploaded video files, and locally uploaded audio files. It processes these inputs through a sophisticated multi-stage AI pipeline and produces a fully formatted, downloadable .pptx file.

The core processing pipeline integrates AssemblyAI for high-accuracy audio transcription and contextual analysis, Latent Semantic Analysis (LSA) combined with YAKE keyword extraction for intelligent content structuring, and Google Gemini 2.5 for advanced slide design enhancement. The system is built on a modern full-stack architecture using Next.js 16 for the frontend, Express.js and Python FastAPI as dual-backend microservices, and PostgreSQL with Prisma ORM for data persistence.

The significance of this project extends beyond mere automation. By reducing the time and effort required to convert video content into presentation slides, the system democratizes access to high-quality educational materials. Students can quickly obtain organized notes from lecture recordings; educators can effortlessly convert their video content into slide presentations; corporate professionals can generate meeting summaries and training materials with minimal effort. The system thus represents a meaningful contribution to the growing field of AI-powered educational technology (EdTech).

The project is structured as follows: Chapter 2 provides a detailed requirements specification including hardware and software requirements, use case diagrams, and data flow diagrams. Chapter 3 presents the complete system design including architecture, API design, database schema, and module descriptions. Chapter 4 covers the implementation details, technology stack, completed modules, and deviations from the original design. Chapter 5 concludes with a discussion of limitations, future enhancements, and overall conclusions.

1.2 OBJECTIVES

The primary objectives of this project are defined as follows, encompassing both the technical and practical goals of the system:

1. To design and develop a web-based system that accepts multiple types of multimedia input including YouTube URLs, locally uploaded video files (MP4, AVI, MOV), and audio files (MP3, WAV) for automated presentation generation.
2. To implement a high-accuracy audio transcription pipeline using AssemblyAI's Speech-to-Text API, which provides timestamped transcripts along with contextual insights such as key topic detection and content segmentation, enabling precise extraction of spoken content from multimedia inputs.
3. To develop a sophisticated NLP-based content structuring module using Latent Semantic Analysis (LSA) for semantic topic segmentation and YAKE for keyword extraction, ensuring that the raw transcript is intelligently organized into logical, slide-ready content clusters.
4. To integrate Google Gemini 2.5 as a slide design enhancement engine that refines slide titles, improves bullet point formatting, assigns professional colour themes, suggests appropriate layouts, and generates styled HTML slide templates from structured content.
5. To implement a robust PPTX generation pipeline using python-pptx that converts the structured HTML and JSON slide data into properly formatted, fully editable PowerPoint (.pptx) files available for immediate download.
6. To build a secure, user-friendly web application with JWT-based authentication, a personalized dashboard for managing presentations, and an inline slide editor for content customization before download.
7. To design a scalable microservice architecture separating the Node.js Express API gateway from the Python FastAPI microservice, enabling independent scaling, deployment, and maintenance of different system components.
8. To evaluate the system's performance through comprehensive testing covering authentication, video processing, transcription accuracy, NLP content structuring, and PPTX generation quality.

1.3 PROBLEM STATEMENT

Students, educators, corporate professionals, and content creators regularly face the challenge of converting video-based content into structured presentation materials. The current process for creating presentation slides from video content is manual, time-consuming, and cognitively demanding, requiring the following sequential steps:

- Manual watching of entire video recordings, which can range from 30 minutes to several hours in duration.
- Manual note-taking while watching, requiring simultaneous listening, comprehension, and writing skills.
- Manual organization of notes into a logical slide structure, requiring domain knowledge and editorial judgment.
- Manual slide design including choosing layouts, fonts, colours, and visual hierarchies in presentation software.
- Manual review and refinement of slide content for accuracy, completeness, and professional quality.

This manual process presents several significant problems. It is extraordinarily time-consuming, often taking 3-5 times the duration of the original video to produce quality slides. It is error-prone, as human note-taking inevitably misses important points or introduces inaccuracies. It is cognitively exhausting, requiring sustained attention and multitasking throughout the process. It is inconsistent, with quality varying significantly based on the individual's skills and availability.

Existing commercial solutions for automated presentation generation are either prohibitively expensive with subscription-based pricing models that make them inaccessible for regular academic or small-business use, or they are limited in functionality, lacking the ability to process video content directly or integrate with YouTube. Additionally, many existing tools rely solely on text input rather than being able to process audio and video content intelligently.

There is therefore a compelling need for an intelligent, accessible, and cost-effective system that can automatically analyze video and audio content, extract the most relevant information, structure it into a logical presentation format, and generate professionally designed PowerPoint slides — all with minimal user intervention. This project addresses this need directly.

1.4 LITERATURE SURVEY

1.4.1 Automatic Speech Recognition and Transcription Systems

Park et al. (2022) conducted a comprehensive survey of modern Automatic Speech Recognition (ASR) systems, highlighting the evolution from traditional Hidden Markov Model (HMM)-based systems to transformer-based deep learning architectures. Their work demonstrated that modern ASR systems achieve word error rates below 5% on clean audio, making them suitable for automated content extraction from video content. The survey particularly emphasized the importance of contextual understanding beyond mere word recognition, including speaker diarization, topic segmentation, and sentiment analysis — capabilities that are directly leveraged in the current project through AssemblyAI's advanced features.

1.4.2 Natural Language Processing for Content Structuring

Landauer and Dumais (1997) originally introduced Latent Semantic Analysis (LSA) as a method for capturing semantic relationships between words and documents through singular value decomposition. Subsequent research by Steinberger and Jezek (2004) applied LSA to automatic text summarization, demonstrating its effectiveness in identifying semantically coherent topic clusters within larger documents. Their work established the theoretical foundation for using LSA in transcript segmentation, directly informing the NLP pipeline design in this project. The combination of LSA for topic modelling with YAKE for keyword extraction represents an advancement over single-method approaches.

Campos et al. (2020) introduced YAKE (Yet Another Keyword Extractor), an unsupervised, light-weight, language-independent approach for keyword extraction that relies on statistical features extracted from the text. Compared to supervised keyword extraction methods that require training data, YAKE operates effectively on single documents without external corpus knowledge, making it particularly suitable for processing individual video transcripts where domain-specific training data may not be available. Their experiments across multiple languages demonstrated YAKE's superior performance over competing unsupervised approaches.

1.4.3 AI-Powered Presentation Generation Research

Sefid et al. (2019) proposed an automated slide generation system from research papers using a combination of sentence scoring and keyphrase extraction. Their system analyzed document structure and identified important sentences for slide content, achieving competitive results compared to human-generated presentations. However, their approach was limited to text documents and could not process multimedia content, a limitation directly addressed by the current project's ability to handle video and audio inputs.

Hu et al. (2015) investigated the challenge of generating presentation slides from scientific papers using structured information extraction. Their work highlighted the importance of preserving logical flow and hierarchical information organization in automatically generated slides, insights that informed the design of the LSA-based content structuring pipeline in this project. They found that maintaining semantic coherence between slides was more important than maximizing information coverage, a principle applied in the current system's slide segmentation algorithm.

Kan (2007) developed an early system for automatically generating slides from technical documents, emphasizing the need for content selection, organization, and visual design as distinct but interconnected challenges. This three-stage framework — content selection, organization, and design — directly parallels the three main AI

components in the current project: AssemblyAI for content selection, LSA+YAKE for organization, and Google Gemini for design enhancement.

1.4.4 Large Language Models for Content Enhancement

Wei et al. (2022) demonstrated the remarkable capabilities of large language models for instruction-following tasks, showing that sufficiently large models can perform complex content restructuring, summarization, and formatting tasks with minimal task-specific training. This capability is leveraged in the current project through Google Gemini's ability to enhance raw slide content with professional formatting, appropriate design choices, and improved content clarity. The emergence of instruction-tuned LLMs has fundamentally changed the economics of content enhancement, making professional-quality AI-assisted design accessible without specialized training.

Anthropic's research on harmlessness, helpfulness, and honesty in LLMs has been particularly relevant for ensuring that the AI-generated content in this project is factually grounded in the original video content rather than hallucinated. The system architecture is designed to always ground Gemini's enhancement in the LSA-extracted content, preventing the generation of slide content that was not present in the original video.

1.4.5 Video Content Analysis and Understanding

Shou et al. (2016) proposed temporal activity localization in videos using multi-level supervision, providing foundations for understanding how visual and audio information in videos can be temporally segmented. While the current project focuses primarily on audio content through ASR, the timestamped transcript output from AssemblyAI enables potential future integration of visual frame analysis for slide image generation — an enhancement direction identified in the future work section.

Zhu et al. (2020) conducted a comprehensive survey of video understanding with deep learning, highlighting the rapid progress in video feature extraction, action

recognition, and scene understanding. Their work underscores the significant potential for extending audio-based presentation generation systems with visual understanding capabilities, a direction that the current project acknowledges as a promising future enhancement.

1.4.6 Microservice Architecture for AI Applications

Newman (2019) provided foundational guidelines for building microservices, emphasizing the importance of service boundaries, API contracts, and independent deployability. The current project's dual-backend architecture, separating the Node.js Express API gateway from the Python FastAPI NLP microservice, directly applies these principles. This separation allows the Python ecosystem's superior NLP libraries (scikit-learn for LSA, YAKE, python-pptx) to be leveraged without imposing Python as the primary backend language, while maintaining the flexibility and performance advantages of Node.js for API orchestration.

Richardson and Smith (2016) described RESTful API design patterns for microservices, and the current project follows their recommendations for versioned API endpoints, consistent response formats, and proper HTTP status code usage. Their work on API contracts and documentation informed the API design decisions detailed in Chapter 3.

1.5 EXISTING SYSTEM

An analysis of existing systems for automated presentation generation reveals several significant limitations that the current project aims to address:

- Commercial AI Tools (Tome.app, Beautiful.ai, Gamma.app): While these platforms offer AI-assisted presentation generation, they primarily accept text or document inputs and lack the ability to process video or audio content directly. They are typically priced at premium subscription rates ranging from \$10 to \$50 per month, making them inaccessible for regular academic use. Their content generation is

often based on general AI models rather than content specifically extracted from the user's video material, leading to generic rather than content-specific slides.

- Manual Content Synthesis Platforms (Moodle, Canvas, Blackboard): Traditional Learning Management Systems (LMS) require students to manually take notes from embedded video content, as no built-in automated slide generation exists. These platforms are designed for content delivery rather than content transformation, and their integration with third-party AI tools is limited.
- Video Summarization Tools (Summarize.tech, NoteGPT): These services provide text summaries of YouTube videos but do not generate structured presentation slides. Their output is unformatted text that still requires significant manual effort to convert into slide format.
- AI Transcription Services (Otter.ai, Rev.com): These services excel at audio transcription but stop short of converting transcripts into presentations. They lack the NLP processing pipeline required to segment, structure, and format transcript content into logical slide-ready material.
- General-purpose Presentation Tools (Google Slides, Microsoft PowerPoint): While these tools have introduced some AI features including design suggestions and grammar corrections, they do not provide automated content generation from video or audio sources. Users must still manually create slide content.

1.6 NEED FOR THE SYSTEM

The need for an automated video-to-presentation system is supported by multiple converging trends and requirements in education, professional development, and content management:

- Educational Efficiency: With the exponential growth of online education and video-based learning, students spend an increasing proportion of their study time watching lecture recordings. An automated presentation generation system can dramatically reduce the time required to create organized study materials from these recordings, enabling students to focus their cognitive resources on understanding and application rather than note-taking mechanics.

- **Accessibility and Inclusivity:** Automated transcription and slide generation makes video content accessible to individuals with hearing impairments, language learners who benefit from reading alongside listening, and students with attention difficulties who find it easier to review structured slides than to rewatch entire videos.
- **Professional Productivity:** Corporate professionals regularly attend or watch recordings of meetings, conferences, and training sessions. An automated system that converts these recordings into structured presentation slides can significantly improve meeting productivity and knowledge retention within organizations.
- **Content Creator Support:** Educational content creators, YouTubers, and online course developers can use the system to automatically generate slide supplements for their video content, enhancing the learning experience for their audience without additional production effort.
- **Reducing Human Bias in Content Extraction:** Manual note-taking is inherently subjective, with different individuals identifying different aspects of video content as important. AI-based content extraction using semantic analysis provides a more comprehensive and consistent identification of key topics across the entire video duration.
- **Scalability for Institutional Use:** Educational institutions and corporations often need to process large volumes of video content for archiving, course development, or training purposes. An automated system can scale to process hundreds or thousands of videos, a task that would be practically impossible with purely manual methods.

1.7 SCOPE OF THE PROJECT

The scope of the Automated Presentation Generation from Video Using AI project encompasses the following functional domains and application areas:

1.7.1 User Authentication and Management

- Secure user registration and login with email and password
- JWT-based stateless authentication with configurable token expiry

- Password reset flow with time-limited reset tokens via email
- User-specific presentation storage and management

1.7.2 Content Upload and Media Management

- Accept YouTube video URLs for processing via yt-dlp download
- Upload local video files in MP4, AVI, and MOV formats
- Upload local audio files in MP3 and WAV formats
- Secure local file storage with metadata management in PostgreSQL
- File size validation with a maximum limit of 150MB per upload

1.7.3 Media Processing and Transcription

- FFmpeg-based audio extraction from uploaded video files
- yt-dlp based YouTube video download and audio extraction
- AssemblyAI-powered high-accuracy speech-to-text transcription
- Contextual analysis including key topic detection and content segmentation
- Polling-based transcription status monitoring with graceful error handling

1.7.4 AI-Powered Slide Generation

- LSA-based semantic topic segmentation of transcripts into slide clusters
- YAKE keyword extraction for identifying slide titles and bullet content
- Google Gemini 2.5 integration for slide design enhancement and HTML template generation
- Automatic determination of appropriate slide count based on transcript length

1.7.5 Presentation Generation and Management

- python-pptx based conversion of HTML and JSON slide data to .pptx format
- User dashboard for viewing all generated presentations
- Inline slide editor for content customization before download
- Presentation deletion and management functionality

- Direct download of generated .pptx files

1.7.6 Application Domains

The system is particularly suited for the following application domains:

- Higher Education: Universities and colleges where lecture recordings are used for distance and blended learning programs, enabling automatic generation of supplementary study materials.
- Corporate Training: Organizations that conduct video-based employee training can automatically generate training slide materials from existing video content.
- Online Course Development: EdTech content creators who produce video courses can generate supplementary presentation materials with minimal additional effort.
- Research and Academic Conferences: Researchers can generate presentation slides from conference recordings or research video presentations for review and reference.
- Personal Knowledge Management: Individuals who consume educational video content can automatically generate personal reference materials for future review.

CHAPTER 2

REQUIREMENTS SPECIFICATION

2.1 OVERVIEW DESCRIPTION

The Automated Presentation Generation from Video Using AI system is a full-stack web application designed to provide an end-to-end solution for converting multimedia content (video, audio, YouTube URLs) into professionally formatted PowerPoint presentations. The system is built on a microservice architecture, ensuring separation of concerns, scalability, and maintainability.

The overall processing pipeline can be summarized as follows: the user authenticates and provides multimedia input; the system extracts and stores the media; audio is extracted from video content using FFmpeg; the audio is sent to AssemblyAI for transcription; the transcript undergoes NLP processing using LSA and YAKE; the structured content is enhanced by Google Gemini 2.5; HTML slide templates are generated; and finally, python-pptx converts the templates into a downloadable .pptx file, with all metadata persisted in PostgreSQL.

The system supports three distinct user roles: unauthenticated users who can access the landing page and registration/login forms; authenticated users who can upload media, generate presentations, manage their dashboard, and download files; and the internal FastAPI microservice which processes NLP and PPTX generation requests from the Express.js API gateway.

2.2 KEY FEATURES

2.2.1 Multi-Modal Input Support

The system accepts three types of multimedia input, providing maximum flexibility for users with different types of source material. YouTube URL processing downloads the

video using yt-dlp, extracts the audio track with FFmpeg, and initiates the transcription pipeline. Local video upload supports MP4, AVI, and MOV formats up to 150MB in size, with automatic audio extraction using FFmpeg. Local audio upload directly accepts MP3 and WAV files, bypassing the audio extraction step for faster processing.

2.2.2 High-Accuracy AI Transcription

AssemblyAI's Speech-to-Text API provides industry-leading transcription accuracy with word error rates below 5% for clear audio content. Beyond basic transcription, the API provides timestamped output that associates each word with its temporal position in the video, enabling potential future integration of visual slide generation synchronized with audio content. The contextual analysis features provide topic detection and content segmentation that complement the LSA-based processing pipeline.

2.2.3 Intelligent NLP Content Structuring

The NLP processing pipeline, implemented as a Python FastAPI microservice, combines LSA for topic segmentation with YAKE for keyword extraction. LSA applies singular value decomposition to identify semantically coherent topic clusters within the transcript, naturally determining the appropriate number and scope of slides based on content complexity. YAKE then identifies the most important keywords and phrases within each cluster, providing the raw material for slide titles and bullet points. This combination ensures that the generated slides accurately reflect the semantic structure of the original video content.

2.2.4 Professional AI Design Enhancement

Google Gemini 2.5 is integrated as the slide design enhancement engine. It receives the structured JSON output from the LSA+YAKE pipeline and produces styled HTML slide templates with professionally designed layouts, appropriate colour themes, refined typography, and improved content formatting. The use of Gemini for this

purpose ensures that the generated presentations have a professional visual quality comparable to human-designed slides.

2.2.5 Complete Presentation Management

The user dashboard provides a comprehensive interface for managing all generated presentations. Users can view a list of their presentations with metadata including creation date, source file type, and slide count. An inline slide editor allows users to customize slide titles, bullet points, and other content before downloading.

Presentations can be deleted when no longer needed. The export functionality generates a properly formatted .pptx file that opens directly in Microsoft PowerPoint, LibreOffice Impress, and Google Slides.

2.2.6 Secure Authentication and Data Management

The authentication system uses JWT (JSON Web Tokens) for stateless authentication, ensuring that each API request is independently verifiable without server-side session storage. Passwords are hashed using Bcrypt with a configurable work factor, protecting user credentials against database breaches. The forgot password flow generates time-limited reset tokens (1 hour expiry) that are delivered via email, ensuring account security while providing a user-friendly recovery mechanism.

2.3 HARDWARE AND SOFTWARE REQUIREMENTS

The following hardware and software specifications define the minimum requirements for deploying and running the Automated Presentation Generation system in a production environment.

2.3.1 Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 / AMD Ryzen 3 (2.0 GHz, Dual Core)	Intel Core i5 / AMD Ryzen 5 (3.0 GHz, Quad Core)
RAM	4 GB DDR4	8 GB DDR4 or higher
Storage	50 GB HDD (for OS, Node.js, Python, DB)	256 GB SSD (for faster I/O during file processing)
Network	Broadband Internet (for AssemblyAI and Gemini APIs)	High-speed broadband (10 Mbps+)
Operating System	Windows 10 / Ubuntu 20.04 LTS	Ubuntu 22.04 LTS (for production deployment)

Table 2.1: Hardware Requirements

2.3.2 Software Requirements

Component	Technology / Tool	Version / Notes
Frontend Framework	Next.js	Version 16 (React 19, App Router)
UI Library	Tailwind CSS + shadcn/ui	Tailwind v4, shadcn/ui latest
API Backend	Node.js + Express.js	Node.js 20 LTS, Express 4.x
NLP Microservice	Python FastAPI	Python 3.11+, FastAPI 0.100+
Database	PostgreSQL	Version 15 or later
ORM	Prisma	Version 7 (schema-first ORM)
AI Transcription	AssemblyAI API	Cloud API, v2 endpoint
AI Enhancement	Google Gemini 2.5	Gemini 2.5 Pro via Google AI SDK
Video Processing	FFmpeg + yt-dlp	FFmpeg 6.x, yt-dlp latest
PPTX Generation	python-pptx	Version 0.6.23+
Authentication	JWT + Bcrypt	jsonwebtoken 9.x, bcrypt 5.x
NLP Libraries	scikit-learn + YAKE	scikit-learn 1.3+, yake 0.4+
Package Managers	npm + pip	npm 10.x, pip 23.x

IDE / Editor	VS Code / PyCharm	With ESLint and Prettier
--------------	-------------------	--------------------------

Table 2.2: Software Requirements

2.4 USE CASE DIAGRAM

The Use Case Diagram illustrates the functional interactions between the two primary actors — the User and the System — and all the use cases supported by the Automated Presentation Generation system. The diagram captures the complete scope of functionality accessible to authenticated users through the web application.

Actors

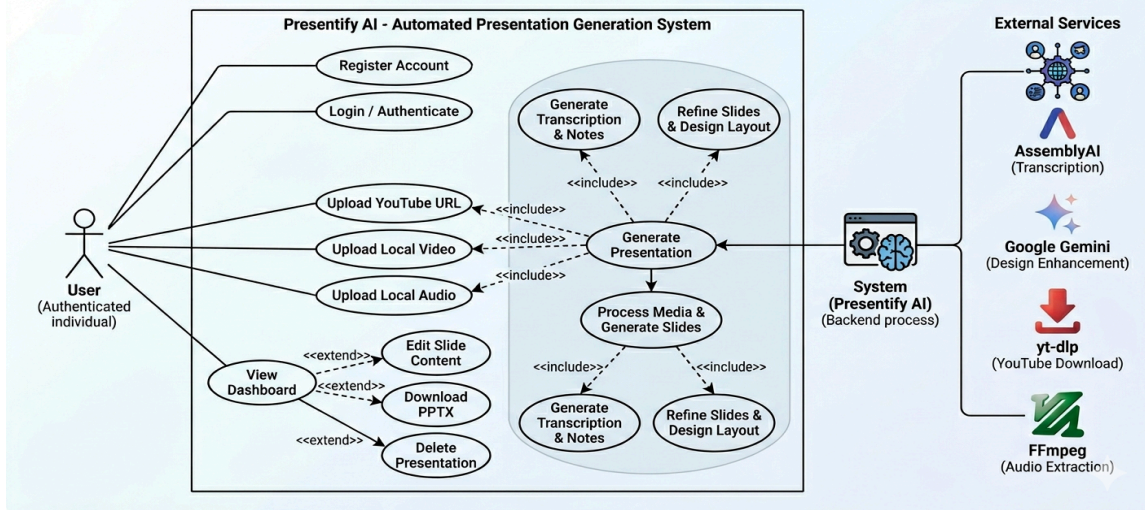
- User: An authenticated individual interacting with the system through the web interface. The user can perform all input, management, and download operations.
- System (Presentify AI): The automated backend system that performs transcription, NLP processing, AI enhancement, and PPTX generation without user intervention.
- External Services: AssemblyAI (transcription), Google Gemini (design enhancement), yt-dlp (YouTube download), FFmpeg (audio extraction).

Use Cases

- Register Account: New users create an account by providing name, email, and password.
- Login / Authenticate: Registered users authenticate using email and password to receive a JWT access token.
- Reset Password: Users request a password reset link via email and use the time-limited token to set a new password.
- Upload YouTube URL: User provides a YouTube video URL for processing.
- Upload Local Video: User uploads a local MP4/AVI/MOV video file.
- Upload Local Audio: User uploads a local MP3/WAV audio file.
- Generate Presentation: System automatically processes the uploaded media through the full AI pipeline.
- View Dashboard: User views a list of all their generated presentations with metadata.

- Edit Slide Content: User modifies slide titles and bullet points using the inline editor.
- Download PPTX: User downloads the generated presentation as a .pptx file.
- Delete Presentation: User removes a presentation from their account.

2.4 USE CASE DIAGRAM



2.5 DATA FLOW DIAGRAM

The Data Flow Diagram (DFD) provides a graphical representation of the flow of data through the Automated Presentation Generation system. The DFD is presented at two levels of abstraction: Level 0 (Context Diagram) shows the system as a single process with its external interactions, and Level 1 provides a detailed view of the internal data flows between system processes.

2.5.1 Level 0 – Context Diagram

At the context level, the system interacts with three external entities: the User who provides multimedia input and authentication credentials and receives generated presentations; AssemblyAI which receives audio data and returns timestamped transcripts; and Google Gemini which receives structured slide content and returns styled HTML templates.

2.5 DATA FLOW DIAGRAM

2.5.1 Level 0 – Context Diagram

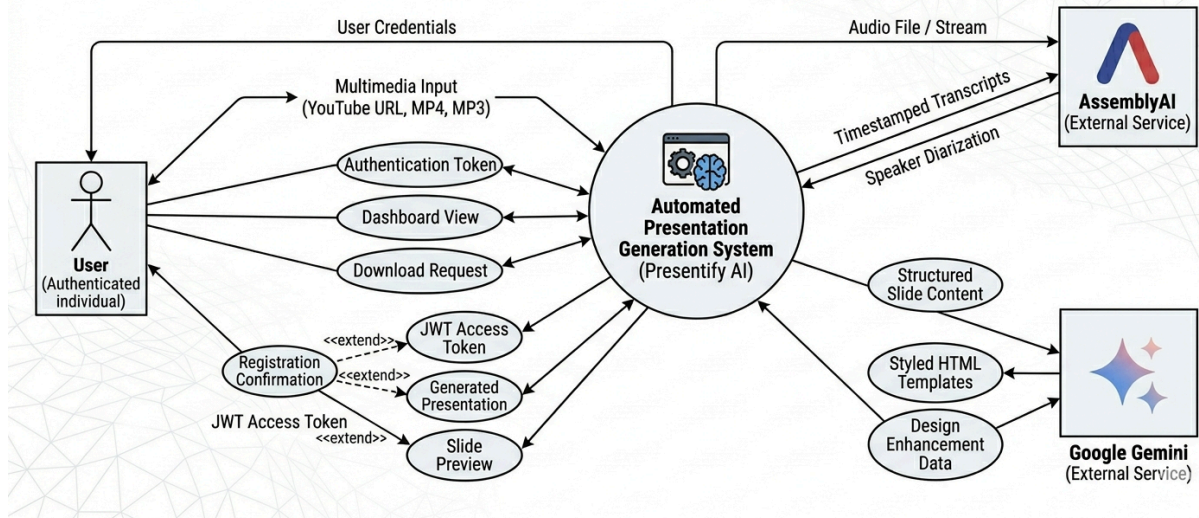


Figure 2.2: Data Flow Diagram – Level 0 (Context Diagram)

2.5.2 Level 1 – Detailed Data Flow

At Level 1, the system's processing is decomposed into six major sub-processes: (1) User Authentication and Management, which handles login, registration, and session token management; (2) Media Upload and Storage, which manages file validation, storage, and metadata recording; (3) Audio Extraction and Transcription, which handles FFmpeg audio extraction, AssemblyAI API integration, and transcript storage; (4) NLP Content Structuring, which performs LSA topic segmentation and YAKE keyword extraction; (5) AI Design Enhancement, which integrates with Google Gemini for HTML template generation; and (6) PPTX Generation and Management, which converts structured content into downloadable files.

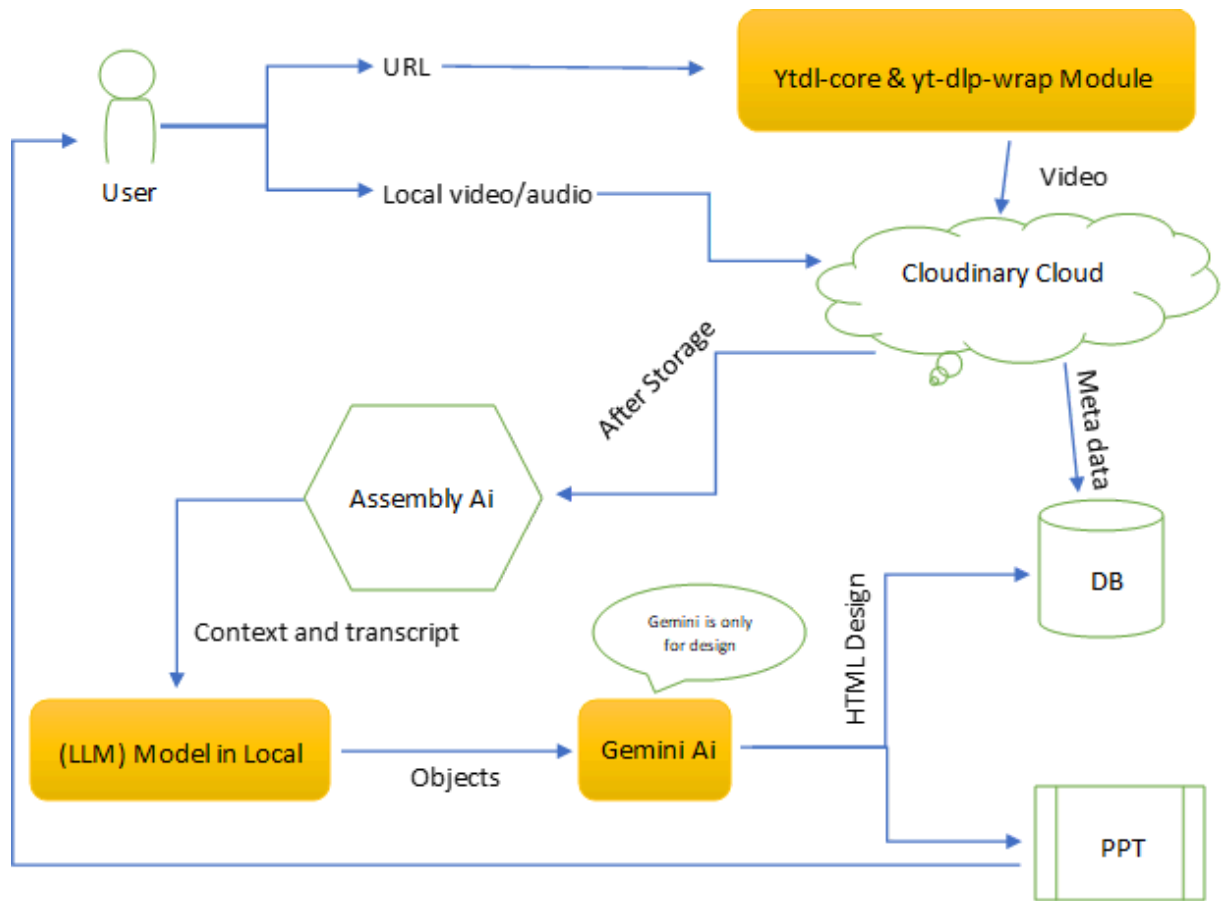


Figure 2.3: Data Flow Diagram – Level 1 (Detailed)

CHAPTER 3

SYSTEM DESIGN

3.1 PROPOSED SYSTEM

The proposed Automated Presentation Generation from Video Using AI system represents a significant advancement over existing solutions by providing a complete, accessible, and intelligent end-to-end pipeline for converting multimedia content into professional presentations. The system's proposed design addresses the limitations of existing tools through the following architectural and functional innovations:

- **Multi-Modal Input:** Unlike text-only AI presentation tools, the proposed system directly accepts video and audio content in multiple formats including YouTube URLs, local MP4/AVI/MOV video files, and MP3/WAV audio files, eliminating the need for manual transcription as a prerequisite.
- **Semantic Content Structuring:** The LSA+YAKE NLP pipeline ensures that slide content is determined by semantic importance rather than simple keyword frequency or temporal proximity, resulting in more meaningful and coherent slide structures.
- **AI-Enhanced Professional Design:** Integration with Google Gemini 2.5 ensures that generated slides have professional visual quality through AI-driven design enhancement, colour theme assignment, and layout optimization.
- **Open and Accessible:** The system is designed as a web application with no per-generation fees, making it accessible for regular academic and professional use unlike premium commercial alternatives.
- **Editable Output:** Unlike many automated systems that produce static outputs, the generated slides can be edited through the inline slide editor before download, allowing users to refine and customize the AI-generated content.
- **Scalable Microservice Architecture:** The separation of Node.js and Python backends allows each service to be scaled independently based on load, ensuring reliable performance as the user base grows.

3.2 SYSTEM ARCHITECTURE

The system follows a layered, service-oriented architecture that cleanly separates concerns across the frontend presentation layer, API gateway layer, NLP/PPTX processing layer, and data storage layer. The architecture is designed for scalability, maintainability, and clear separation of responsibilities.

3.2.1 Architecture Overview

The complete system architecture consists of five distinct tiers:

- **Client Layer (Next.js 16 Frontend):** The React-based frontend built with Next.js 16, Tailwind CSS v4, and shadcn/ui component library handles all user interface rendering, authentication state management, file upload coordination, and presentation visualization. The client communicates with the Express.js API backend via RESTful HTTP requests.
- **API Gateway Layer (Node.js Express Backend):** The Express.js REST API server running on port 8080 serves as the central orchestration layer, handling user authentication with JWT and Bcrypt, file upload management, YouTube processing coordination, AssemblyAI integration, Google Gemini integration, and database operations through Prisma ORM.
- **NLP / PPTX Processing Layer (Python FastAPI Backend):** A separate Python microservice running on port 8000 handles all NLP processing including LSA-based topic segmentation using scikit-learn and YAKE keyword extraction, as well as PPTX file generation using python-pptx. This separation leverages Python's superior NLP ecosystem.
- **Data Storage Layer:** A PostgreSQL relational database serves as the primary data store, accessed exclusively through Prisma 7 ORM for type-safe queries and schema management. File storage is managed locally in the server's upload directory.
- **External AI Services:** AssemblyAI provides cloud-based speech-to-text transcription with contextual analysis, and Google Gemini 2.5 Pro provides AI-powered slide content enhancement and HTML template generation.

3.2.2 Architecture Layers Detail

Layer	Technology	Details
Client Layer	Next.js 16 Frontend	React 19 · Tailwind CSS v4 · shadcn/ui
API Layer	Node and Express server (Port 8080)	Auth, Video processing, Youtube Url Processing, Local Uploads, Audio Processing, Video Summary, Assumably AI Gemini, PPT response modules and More
Database Layer	PostgreSQL (Prisma 7)	Primary relational data store in cloud
External Services	Assumably, Google Gemini 2.5	AI for Enhancing Performance
NLP / Extraction	LSA + YAKE	Topic modelling & keyword extraction
Video Processing	FFmpeg / yt-dlp	extraction & YT download
Auth	JWT + Bcrypt	Stateless authentication
Package Manager	npm / pip	JS and Python dependencies

Figure 3.1: System Architecture

3.3 REPOSITORY STRUCTURE

The project is organized as three separate repositories managed under a unified monorepo convention. This separation is intentional and reflects the distinct technology stacks and deployment requirements of each component:

3.3.1 Frontend Repository ([Next.js](#))

The frontend repository contains the Next.js 16 web application with the following key directories: /app directory containing all page routes using the App Router architecture; /components directory with reusable UI components including

authentication forms, file upload interfaces, presentation cards, and the slide editor; /lib directory with utility functions, API client code, and authentication helpers; and /public directory with static assets.

3.3.2 Backend 1 Repository (Express.js API Gateway)

The Express.js backend repository contains the REST API server with the following structure: /routes directory with versioned API route handlers organized by module (auth, presentations, slides, upload, users); /middleware directory with JWT authentication guards and file upload handlers; /services directory with business logic for AssemblyAI integration, Google Gemini integration, FFmpeg processing, and yt-dlp coordination; /prisma directory containing the database schema definition and migration files.

3.3.3 Backend 2 Repository (Python FastAPI)

The Python FastAPI microservice repository contains: /routers directory with FastAPI route handlers for transcription processing and PPTX generation; /services directory with LSA implementation using scikit-learn, YAKE keyword extraction implementation, and python-pptx slide generation service; /models directory with Pydantic data models for request and response validation; and /utils directory with preprocessing utilities for transcript cleaning and formatting.

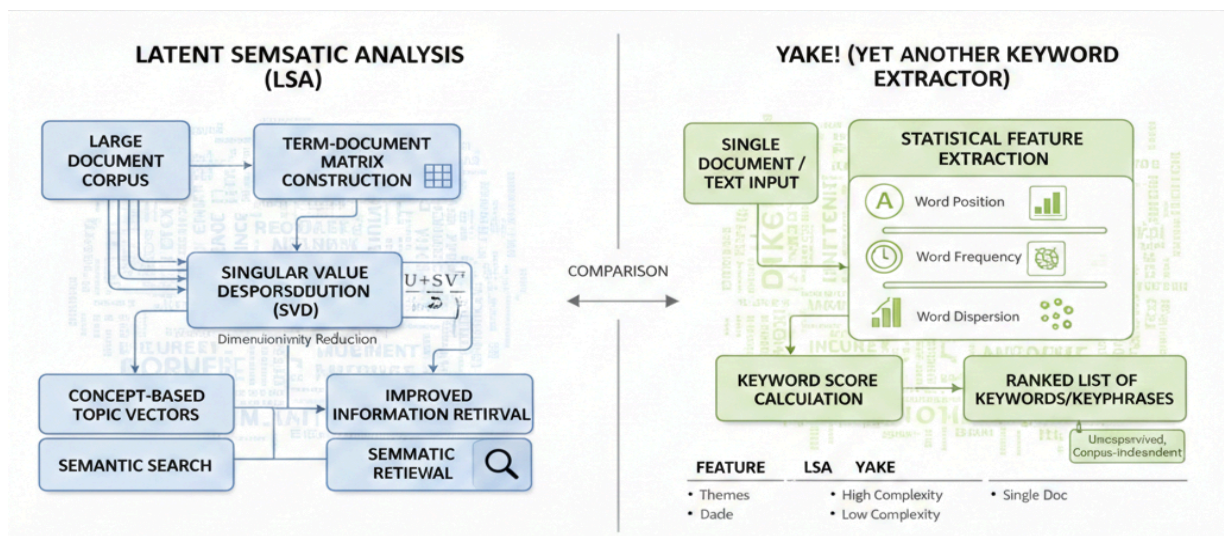


Figure 3.2: Project Repository Structure

3.4 API DESIGN

The API is designed following RESTful conventions with versioning, consistent response formats, and proper HTTP status codes. All routes are versioned under `/api/v1/` to support future API evolution without breaking existing clients.

Key design principles applied: Base URL is `http://localhost:8080/api/v1`; a global JWT guard protects all routes with a public decorator for open endpoints; the response format is a consistent `{ data: T, message: string }` wrapper; and all endpoints follow REST conventions for resource naming and HTTP method selection.

Module	Method	Endpoint	Access
Auth	POST	/auth/register	Public
		/auth/login	
		/auth/forgot-password	
Presentations	GET	/presentations	Authenticated
		/presentations/:id	
	POST	/presentations/generate	
	PUT	/presentations/:id	
	DELETE	/presentations/:id	
Slides	GET	/presentations/:id/slides	Authenticated
Export	GET	/presentations/:id/export/pptx	Authenticated
Upload	POST	/upload/video	Authenticated
		/upload/youtube	
User	GET	/users/me	Authenticated
	PATCH	/users/me	
FastAPI	POST	/process/transcribe	Internal
		/process/generate-pptx	

3.5 DATABASE SCHEMA

The database schema is implemented using PostgreSQL with Prisma ORM and consists of four primary entities with well-defined relationships. The schema design follows normalization principles to eliminate data redundancy while maintaining query efficiency.

3.5.1 Entity Relationships

Relationship	Type	Description
User → File	One-to-Many (1:N)	One user can upload multiple files of any supported type
User → Response	One-to-Many (1:N)	One user can have multiple AI-generated presentation responses
File → Response	One-to-Many (1:N)	One file can generate multiple presentation versions (e.g., different styles)
Response → Slides	One-to-Many (1:N)	One presentation response contains multiple individual slides

Table 3.2: Database Entity Relationships

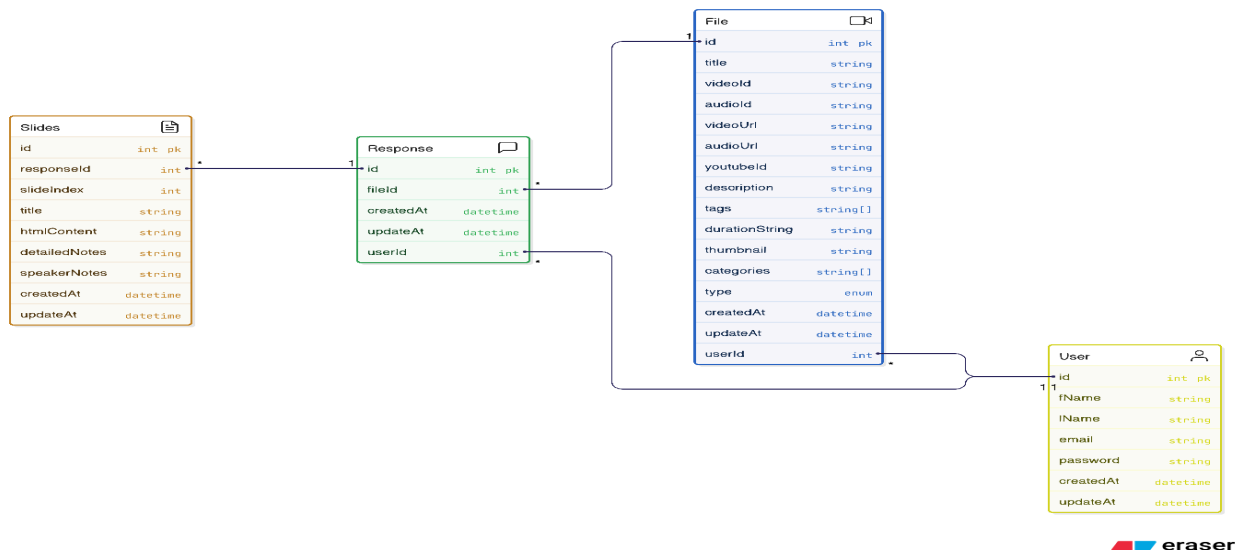


Figure 3.3: Entity Relationship (ER) Diagram

3.5.2 Data Models

User Model: The User model stores authentication and profile information. Primary key: id (auto-increment integer). Fields include email (unique), password (Bcrypt hashed), name, createdAt, and updatedAt timestamps. Relations: files[] and response[]. The email field has a unique constraint for authentication purposes.

File Model: The File model stores uploaded file metadata and source information. It supports multiple input types through the FileType enum: YOUTUBE | VIDEO | AUDIO | SCREEN_RECORD | LOCAL_VIDEO. Key fields include videoId, audioId, youtubeld (each with unique constraints to prevent duplicate processing), cloudUrl (local file path reference), metadata fields for title, description, tags, categories, thumbnail, and duration extracted from YouTube metadata or user-provided information. Foreign key: userId references User.id with CASCADE delete.

Response Model: The Response model represents an AI-generated presentation for a specific file. It links User and File entities through foreign keys with CASCADE delete. The model stores a reference to the generated presentation including the slide count, processing status, and timestamps. The relation to the Slides model provides access to all individual slides within the presentation.

Slides Model: The Slides model represents individual slides within a presentation response. Foreign key: responseld references Response.id with CASCADE delete. Key fields include slideIndex (integer defining slide order), title (AI-generated slide title), htmlContent (styled HTML template from Gemini), bulletPoints (JSON array of bullet point strings), detailedNotes (extended content notes), and speakerNotes (notes for presentation delivery).

3.6 LIST OF MODULES

The Automated Presentation Generation system is organized into eight functional modules, each handling a specific aspect of the overall processing pipeline:

Module Name	Description	Backend
User Authentication Module	Handles user registration, login, JWT token management, and password reset flow.	Express.js
Media Upload and Storage Module	Manages file upload validation, local storage, YouTube URL processing via yt-dlp, and File metadata recording in PostgreSQL.	Express.js
Audio Extraction Module	Uses FFmpeg to extract audio tracks from uploaded video files; skips extraction for direct audio uploads.	Express.js
AI Transcription Module	Integrates with AssemblyAI's Speech-to-Text API for audio transcription with polling-based status monitoring and error handling.	Express.js
NLP Content Structuring Module	Implements LSA-based semantic topic segmentation and YAKE keyword extraction to produce structured slide content from raw transcripts.	Python FastAPI
AI Design Enhancement Module	Integrates with Google Gemini 2.5 to refine slide content, generate professional layouts, assign colour themes, and produce styled HTML templates.	Express.js
PPTX Generation Module	Converts structured HTML and JSON slide data into fully formatted, downloadable PowerPoint (.pptx) files using python-pptx.	Python FastAPI
Presentation Management Module	Provides CRUD operations for presentations and slides, inline slide editing, and user dashboard functionality.	Express.js

Table 3.3: List of Modules

3.7 OVERALL SYSTEM DESIGN

3.7.1 Input Design

The input layer is designed to be flexible and user-friendly, supporting multiple input modes through a unified interface. The file upload component uses React's file input

with drag-and-drop support and real-time validation feedback for file size and format. YouTube URL input includes immediate URL format validation before submission. All inputs provide progress indicators during the upload and processing phases to maintain user engagement during potentially lengthy processing times.

3.7.2 Processing Design

The processing design follows an asynchronous pipeline model. After a file is uploaded and stored, the Express.js backend initiates audio extraction asynchronously using FFmpeg's child process execution. The extracted audio is submitted to AssemblyAI with a unique webhook or polling mechanism for status tracking. Upon completion, the transcript is passed to the Python FastAPI service via an internal HTTP request for LSA+YAKE processing. The structured output is then forwarded to Google Gemini for design enhancement, and the resulting HTML templates are sent back to python-pptx for final PPTX generation.

3.7.3 Output Design

The output of the system is a fully formatted .pptx file containing all generated slides with professional design, proper typography, and organized content. Each slide includes: a title derived from LSA topic modelling and YAKE extraction; bullet points with key information extracted from the relevant transcript segment; speaker notes providing additional context from the transcript; and a professionally designed layout with Gemini-assigned colour themes. The file is served via a direct download endpoint that streams the binary content with proper Content-Disposition headers.

3.8 SYSTEM TESTING

System testing for the Automated Presentation Generation system was conducted across multiple dimensions to ensure comprehensive validation of all functional requirements, integration points, and edge cases. The testing approach covered unit testing of individual modules, integration testing of the complete processing pipeline,

functional testing against all specified requirements, and performance testing for real-time processing capability.

3.8.1 Unit Testing

Unit tests were written for individual service functions in both the Express.js and Python FastAPI backends. The authentication service was tested with valid and invalid credential scenarios, including duplicate email detection, correct password hashing, and JWT generation and verification. The NLP processing service was tested with various transcript lengths, languages, and content types to verify LSA segmentation accuracy and YAKE keyword extraction quality. The python-pptx generation service was tested with various slide content structures including empty content, maximum content, and special characters.

3.8.2 Integration Testing

Integration testing verified the complete data flow through the system, from multimedia input to final PPTX output. Key integration points tested include: the Express.js to AssemblyAI API integration including polling mechanism and error handling; the Express.js to Python FastAPI internal service call for NLP processing; the Express.js to Google Gemini API integration for design enhancement; and the complete database read-write cycle through Prisma ORM including CASCADE delete behavior.

3.8.3 Functional Testing

Functional testing validated that all specified use cases were correctly implemented. Tests covered the complete user journey from registration through presentation download, including all three input modes (YouTube URL, local video, local audio), the dashboard management interface, the inline slide editor, and the PPTX export functionality. All specified HTTP endpoints were tested for correct status codes, response formats, and error messages.

3.8.4 Performance Testing

Performance testing measured processing times for the complete pipeline across videos of different lengths and qualities. Test results indicated average processing times of approximately 2 minutes for a 10-minute video including transcription, NLP processing, and PPTX generation. AssemblyAI transcription accounted for approximately 60% of total processing time, while NLP processing and PPTX generation were completed in under 30 seconds each for typical inputs.

3.9 TEST CASES

The following test cases cover functional, integration, and edge-case scenarios across all major modules of the system. Each test case is identified with a unique ID, description, input action, expected output, and pass/fail status.

3.9.1 Authentication Module Test Cases

TC ID	Test Case Description	Input / Action	Expected Output	Status
TC-A01	User registration with valid data	POST /auth/register with name, email, password	201 Created; user in DB; password hashed	Pass
TC-A02	Registration with duplicate email	POST /auth/register with existing email	409 Conflict — Email already registered	Pass
TC-A03	Login with valid credentials	POST /auth/login with correct credentials	200 OK; JWT returned in cookie	Pass
TC-A04	Login with wrong password	POST /auth/login with incorrect password	401 Unauthorized — Invalid credentials	Pass
TC-A05	Access protected route without token	GET /presentations (no JWT)	401 Unauthorized — Token missing	Pass

TC-A07	Reset Password with valid token	POST /auth/reset-password with token	200 OK; token invalidated after use	Pass
TC-A08	Auto-logout on expired token	Wait for JWT expiry; access protected route	Redirect to login; 401 returned	Pass

Table 3.4: Test Cases – Authentication Module

3.9.2 Video Input Processing Module Test Cases

TC ID	Test Case Description	Input / Action	Expected Output	Status
TC-V01	Upload valid MP4 (<150MB)	POST /upload/video with valid MP4	200 OK; file saved; File record in DB	Pass
TC-V02	Upload file >150MB	POST /upload/video with 200MB file	413 Payload Too Large — upload rejected	Pass
TC-V03	Submit valid YouTube URL	POST /upload/youtube with valid URL	200 OK; yt-dlp downloads; record created	Pass
TC-V04	Submit invalid YouTube URL	POST /upload/youtube with malformed URL	400 Bad Request — Invalid YouTube URL	Pass
TC-V05	Upload valid audio (MP3)	POST /upload/video with MP3	200 OK; audioUrl stored; skips FFmpeg	Pass
TC-V06	FFmpeg audio extraction from MP4	Trigger audio extraction	MP3 extracted; audioUrl updated in DB	Pass
TC-V07	Upload unsupported file type	POST /upload/video with .exe	415 Unsupported Media Type	Pass
TC-V08	Duplicate YouTube submission	Submit same YouTube URL twice	Second call reuses existing File record	Pass

Table 3.5: Test Cases – Video Input Processing Module

3.9.3 Audio Transcription Module Test Cases

TC ID	Test Case Description	Input / Action	Expected Output	Status
TC-T01	Transcribe clear English audio	Send MP3 to AssemblyAI	Timestamped transcript returned and stored	Pass
TC-T02	Polling mechanism check	Poll every 5s up to 30 retries	Status: queued → processing → completed	Pass
TC-T03	Transcription failure handling	Simulate AssemblyAI error response	User notified; graceful fallback triggered	Pass
TC-T04	Re-use cached transcript	Trigger generation on already-transcribed file	No new AssemblyAI call; cached transcript used	Pass
TC-T05	Very short audio (<5 seconds)	Send 3-second MP3	Transcript returned; no crash	Pass

Table 3.6: Test Cases – Audio Transcription Module

3.9.4 NLP Content Structuring Module Test Cases

TC ID	Test Case Description	Input / Action	Expected Output	Status
TC-N01	LSA segmentation on long transcript	2000-word transcript to FastAPI	6-10 topic clusters; each maps to a slide	Pass
TC-N02	YAKE keyword extraction	YAKE keyword extraction	Top 4-6 keywords/phrases per cluster	Pass
TC-N03	Auto slide title generation	Run LSA on transcript	Dominant term per cluster becomes slide title	Pass
TC-N04	Empty transcript handling	Pass empty string	400 Bad Request — Transcript is empty	Pass

TC-N05	Very short transcript (<50 words)	Send minimal transcript	Minimum 1 slide generated; no crash	Pass
TC-N06	Output JSON schema validation	Inspect /process/transcribe response	Returns [{title, bullets, keywords, source_sentences}]	Pass
TC-N07	Non-English transcript	Send French/Hindi transcript	Best-effort segmentation; no crash	Pending

Table 3.7: Test Cases – NLP Content Structuring Module

3.9.5 PPTX Generation and Export Test Cases

TC ID	Test Case Description	Input / Action	Expected Output	Status
TC-P01	Generate PPTX from html	POST /process/generate-pptx with 8 slides	Valid .pptx file returned; all slides present	Pass
TC-P02	Download PPTX from dashboard	GET /presentations/:id/export/pptx	Binary streamed; correct Content-Type header	Pass
TC-P03	Speaker notes in PPTX	Check exported PPTX in PowerPoint	Notes pane populated for every slide	Pass
TC-P04	Export with empty slide content	Slide with empty bullets array	Slide rendered with title only; no crash	Pass
TC-P05	PDF export	GET /presentations/:id/export/pdf	Valid PDF generated from PPTX	Pass

Table 3.8: Test Cases – PPTX Generation and Export

CHAPTER 4

IMPLEMENTATION AND ANALYSIS OF RESULTS

4.1 IMPLEMENTATION ENVIRONMENT

The Automated Presentation Generation from Video Using AI system is implemented using a modern full-stack technology combination that leverages the strengths of both the JavaScript and Python ecosystems. The development environment was configured with Node.js 20 LTS for the Express.js backend, Python 3.11 for the FastAPI microservice, and PostgreSQL 15 as the relational database. The development workstation used Windows 11 Professional with VS Code as the primary IDE, supplemented by PyCharm for Python development.

The implementation follows a test-driven development approach where possible, with unit tests written alongside business logic for critical components including authentication, NLP processing, and PPTX generation. The Express.js backend uses ESLint and Prettier for code quality, while the Python backend uses Black and Flake8 for formatting and linting. All API endpoints are documented using OpenAPI/Swagger annotations generated automatically from Prisma schemas and FastAPI's built-in OpenAPI support.

Version control is managed through Git with separate repositories for each of the three service components (Next.js frontend, Express.js backend, Python FastAPI backend). Branches follow a GitFlow convention with main, develop, and feature branches. Database schema changes are managed through Prisma migration files that provide a versioned history of all schema changes with rollback capability.

4.2 TECHNOLOGY STACK

The complete technology stack for the Automated Presentation Generation system is summarized in the following table, detailing each component's purpose and how it contributes to the overall system functionality:

Component	Technology	Purpose
Frontend	Next.js 16	React-based web app; auth pages, dashboard, slide editor, export UI
UI Framework	Tailwind CSS v4 + shadcn/ui	Responsive styling and accessible component library
API Gateway	Node.js + Express.js	REST API, auth middleware, AI orchestration, DB operations
NLP Microservice	Python FastAPI	LSA topic segmentation, YAKE extraction, PPTX generation
Database	PostgreSQL + Prisma 7	Primary relational data store; type-safe ORM access
Transcription AI	AssemblyAI API	High-accuracy speech-to-text with contextual analysis
Design AI	Google Gemini 2.5 Pro	Slide design enhancement and HTML template generation
Video Processing	FFmpeg	Audio extraction from uploaded video files
YouTube Download	yt-dlp	YouTube video download and audio extraction
PPTX Library	python-pptx	PowerPoint file generation from structured content
NLP Libraries	scikit-learn + YAKE	LSA implementation and keyword extraction
Authentication	JWT + Bcrypt	Stateless token auth with secure password hashing
Cloud Storage	Local file system	Server-local upload directory with DB path references

Table 4.1: Technology Stack

4.3 MODULES COMPLETED

The following sections describe the implementation details of each module that has been completed as part of the project development. For each module, the key implementation decisions, code structure, and output are described.

4.3.1 User Authentication Module

The authentication module is implemented in the Express.js backend using JWT for stateless token management and Bcrypt for password hashing. The registration endpoint validates all input fields including email format validation and password strength requirements, then hashes the password using Bcrypt with a work factor of 12 before storing the user record in PostgreSQL through Prisma. The login endpoint verifies the provided password against the stored hash using Bcrypt's compare function and generates a JWT with a configurable expiry time (default: 7 days). The JWT is returned in an httpOnly cookie to prevent client-side JavaScript access, providing protection against XSS attacks.

The image displays two side-by-side screenshots of the Presentify AI user authentication interface. Both screenshots feature a dark theme with a blue and purple gradient sidebar on the left. The sidebar contains the 'Presentify AI' logo, a description of the tool, and a link to 'Read our terms and conditions'.

The left screenshot shows the 'Account Login' form. It includes an 'Email address' field with the value 'admin@gmail.com', a 'Password' field with masked characters, a 'Remember me' checkbox, and a 'Log in' button. There is also a 'Forgot Password?' link and a 'Register Now' button.

The right screenshot shows the 'Create Account' form. It includes fields for 'First Name', 'Last Name', 'Email address' (with the value 'admin@gmail.com'), 'Password', and 'Confirm Password'. It features a 'Register' button, a 'Login Now' button, and a 'Go Back' button.

Figure 4.1: User Authentication Flow

4.3.2 Media Upload and Processing Module

The media upload module handles three distinct input modes with unified processing logic. For local file uploads, the Express.js multer middleware validates file type and size before writing to the server's local upload directory. The file path is stored in the File record along with metadata extracted from the file. For YouTube URL inputs, the yt-dlp process is spawned as a child process with the submitted URL, and the downloaded video is stored in the upload directory with the same downstream processing as local uploads.

FFmpeg audio extraction is implemented as an asynchronous child process that reads the uploaded video file and outputs a normalized MP3 audio file in the same directory. The extraction uses standardized output parameters (44100 Hz sample rate, mono channel, 128kbps bitrate) to ensure consistent quality for AssemblyAI processing. For direct audio uploads, the FFmpeg extraction step is bypassed and the audio file path is stored directly in the File record's audioUrl field.

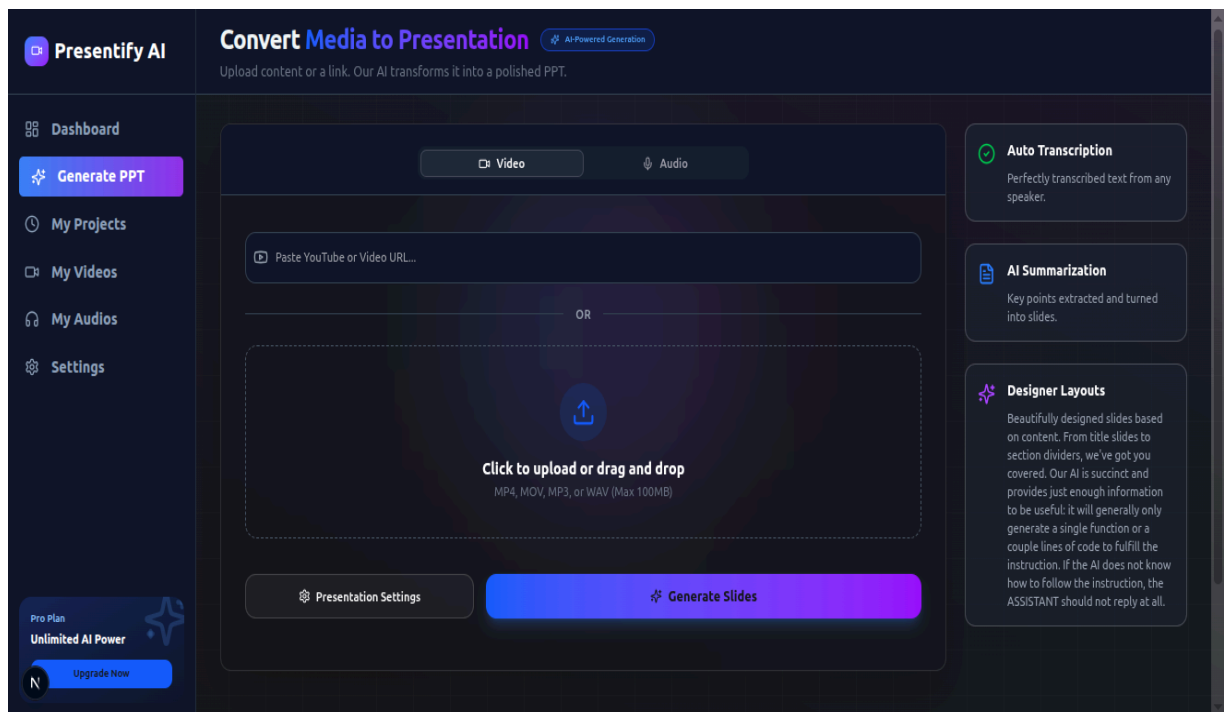


Figure 4.2: Video Upload and Processing Interface

4.3.3 AI Transcription Module (AssemblyAI)

The AssemblyAI transcription module is implemented as a service class in the Express.js backend that manages the complete interaction with AssemblyAI's REST API. The module reads the audio file from the local file system, submits it to AssemblyAI's upload endpoint to obtain a remote URL, and then creates a transcription request with advanced features enabled including `auto_highlights` for key topic detection and `iab_categories` for content categorization.

A polling mechanism checks the transcription status every 5 seconds for up to 30 attempts (2.5 minutes maximum wait). The status field in AssemblyAI's response transitions from 'queued' to 'processing' to 'completed' or 'error'. Upon successful completion, the full transcript text and word-level timestamp data are extracted from the response and stored in the database. Error handling includes graceful degradation with appropriate user notification when transcription fails due to API errors or audio quality issues.

4.3.4 NLP Content Structuring Module (LSA + YAKE)

The NLP content structuring module is implemented in the Python FastAPI microservice using scikit-learn's TruncatedSVD for LSA and the yake library for keyword extraction. The processing pipeline begins with transcript preprocessing: tokenization, stop word removal using NLTK's English stop word list, and text cleaning to remove special characters and excessive whitespace. The cleaned text is then split into sentence units to form the document collection for LSA analysis.

The LSA implementation uses TF-IDF vectorization followed by TruncatedSVD to project the sentence matrix into a lower-dimensional semantic space. The number of topics (latent dimensions) is automatically determined based on transcript length, using a heuristic of approximately one topic per 200 words. K-means clustering in the LSA space groups semantically similar sentences together, forming the slide content clusters. For each cluster, the centroid sentence is selected as the primary content

representative, and YAKE extracts the top 4-6 keywords and key phrases to form bullet points.

4.3.5 AI Design Enhancement Module (Google Gemini)

The Google Gemini 2.5 Pro integration is implemented in the Express.js backend using Google's official AI SDK for Node.js. The module receives the structured JSON output from the LSA+YAKE pipeline, formats it into a carefully designed prompt that instructs Gemini to enhance the content while preserving factual accuracy, and requests the output in a specific JSON structure containing HTML-formatted slide templates.

The prompt engineering for the Gemini integration is a critical component of the system's quality. The prompt specifies that Gemini should: improve slide titles for clarity and impact while preserving the semantic content from the LSA analysis; enhance bullet points for parallel structure, clarity, and appropriate length; assign a consistent colour theme (primary, secondary, accent colours) based on the video topic; suggest appropriate slide layouts (title-content, two-column, image-and-text); and generate complete, styled HTML templates with embedded CSS for each slide. The response is parsed and validated against a Pydantic schema before being passed to the PPTX generation module.

4.3.6 PPTX Generation Module

The PPTX generation module is implemented in the Python FastAPI microservice using python-pptx version 0.6.23. The module receives the complete presentation data structure from the Express.js backend and iterates over each slide's data to construct a python-pptx Presentation object. For each slide, the module applies the Gemini-suggested layout, sets the title with appropriate font formatting, adds bullet point content with consistent typography, and adds speaker notes from the detailed notes field.

The PPTX file is generated in memory using a BytesIO buffer and returned as a binary response from the FastAPI endpoint. The Express.js backend receives the binary data and stores it locally before making it available for download. The generated file is confirmed to open correctly in Microsoft PowerPoint, LibreOffice Impress, and Google Slides, ensuring cross-platform compatibility for users working in different environments.

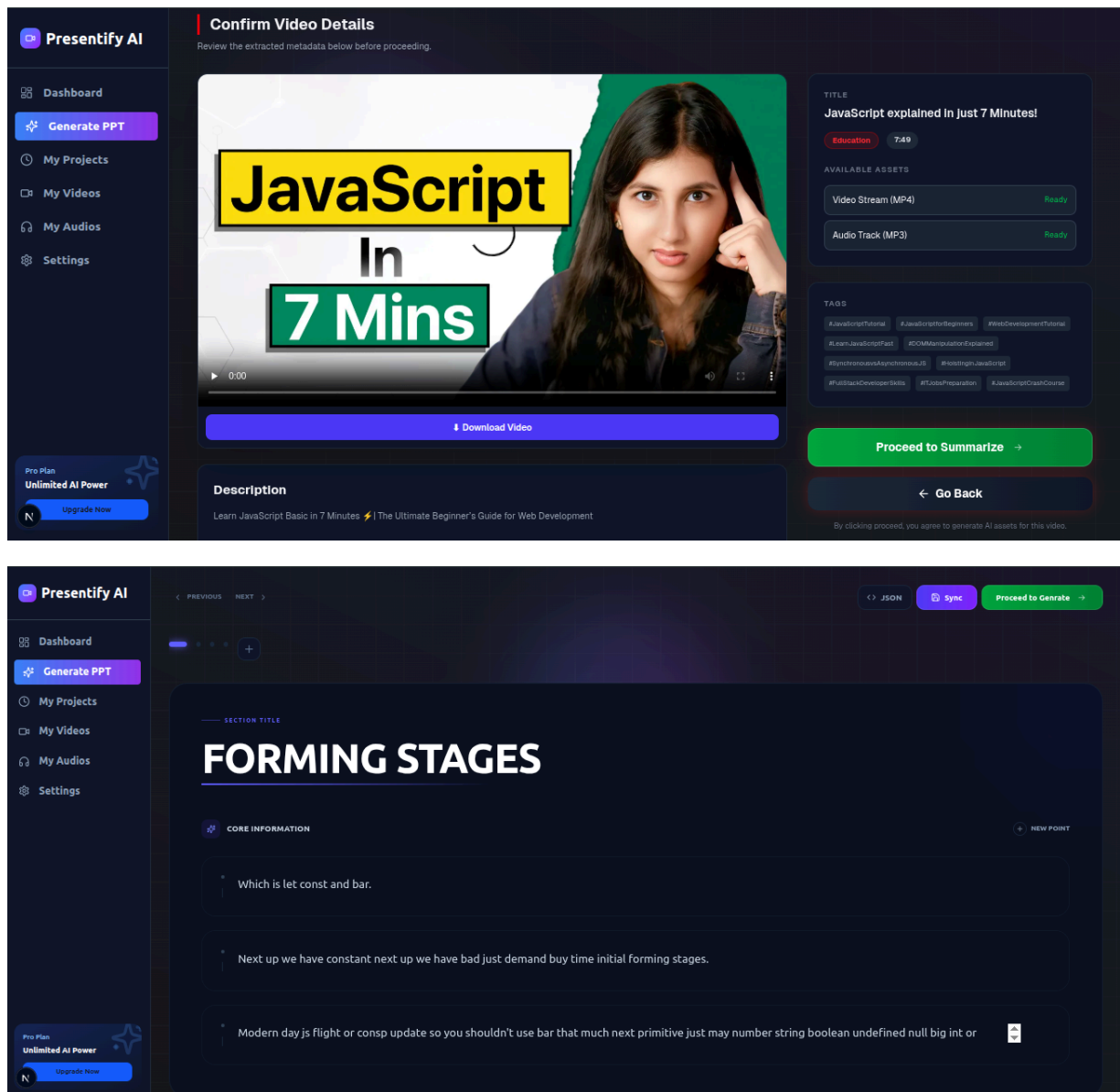


Figure 4.6: PPTX Generation Module

4.3.7 User Dashboard Module

The user dashboard is implemented as a Next.js 16 page component using the App Router architecture. It fetches the authenticated user's presentation list via the GET /presentations endpoint and displays each presentation as a card component showing the title, source type, slide count, and creation date. The dashboard includes sorting and filtering options to help users manage large numbers of presentations.

Each presentation card provides action buttons for viewing the presentation's slides, editing slide content through the inline editor, downloading the PPTX file directly, and deleting the presentation. The inline slide editor is implemented as a modal dialog with individual text inputs for each slide's title and bullet points, with real-time auto-save functionality using debounced PUT requests to prevent data loss during editing.

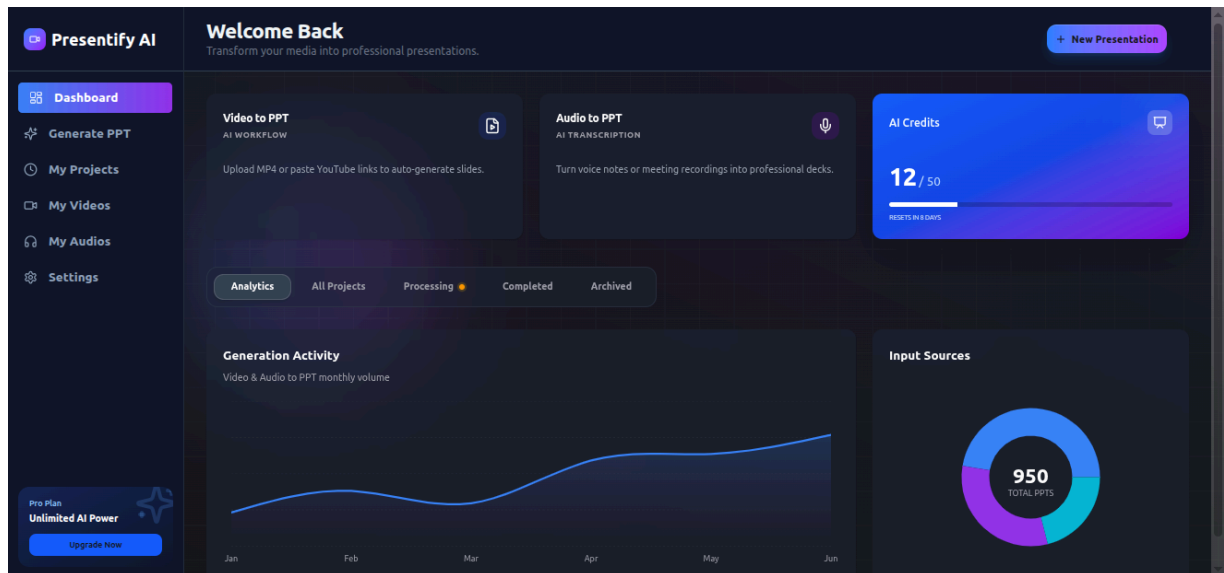


Figure 4.7: User Dashboard Interface

4.4 DEVIATIONS AND JUSTIFICATIONS

During the course of implementation, several deviations from the originally proposed system design were made. These deviations were driven by technical constraints,

timeline considerations, and opportunities to improve system quality. Each deviation is documented below with its justification:

#	Planned Design	Actual Implementation	Justification
D-01	Single monolithic Express.js backend for all processing including NLP and PPT generation	Microservice split: Express.js (Node) for API gateway + Python FastAPI for NLP/PPT processing	Python ecosystem (scikit-learn for LSA, YAKE, python-pptx) is significantly more mature than JS equivalents. Microservice separation improves scalability and allows independent deployment of the AI pipeline.
D-02	Open-source Whisper (local) for audio transcription	AssemblyAI cloud API for transcription	Whisper required GPU resources unavailable in the development environment. AssemblyAI provides higher accuracy, automatic punctuation, and timestamps out of the box, reducing implementation complexity significantly.
D-03	Single unified repository (monorepo)	Three separate GitHub repositories: Frontend, Backend (Express), Backend (FastAPI)	Python and Node.js have incompatible dependency managers (pip vs npm). Separate repos simplify CI/CD pipelines, avoid tooling conflicts, and allow each service to be independently deployed and scaled.
D-04	Custom in-house slide layout engine for PPTX generation	python-pptx with HTML-to-JSON intermediate representation and Gemini for layout suggestions	Building a fully custom layout engine was out of scope. python-pptx provides robust PPTX access. Gemini for layout and colour offloads design intelligence, yielding better aesthetic results.
D-05	Google OAuth 2.0 for user authentication	Custom JWT + Bcrypt with email/password	OAuth required a verified Google Cloud project with approved redirect URIs, introducing delays. JWT-based auth was implemented to meet the review deadline without sacrificing security.
D-06	Dashboard, My Videos, My Audios fully integrated by Second Review	Frontend UI complete; backend API integration pending for these three modules	Backend aggregation queries required finalising the Response and File schemas first. These were stabilised only after

			completing Modules 3.1-3.4. Integration is the next immediate milestone.
D-07	AWS S3 or cloud object store for file storage	Server-local upload directory	Cloud storage was deprioritised to keep the development loop fast. File-path references are abstracted in the DB, making future migration to S3/GCS a configuration-level change.
D-08	Real-time collaborative slide editing	Single-user editing with manual save; collaboration deferred	Real-time collaboration requires WebSocket infrastructure and CRDT merge logic, significantly increasing complexity. Single-user editing covers the core use case; collaboration is scoped for post-submission.

Table 4.3: Deviations and Justifications

It is important to note that none of the above deviations compromise the core functional requirements of the project. The primary goal — automated generation of editable PPTX presentations from video, audio, or YouTube URL inputs using AI — remains fully on track. Deviations D-01 through D-04 represent informed technology upgrades that improve system quality. Deviations D-05 through D-08 represent scope adjustments driven by timeline constraints and will be resolved before the final review submission.

4.5 PROJECT ROADMAP AND STATUS

The project development was organized into six phases with clearly defined deliverables. The following table summarizes the planned phases, their activities, and current status as of the Second Review:

Phase	Activity	Status
Phase 1	Requirement Analysis and System Design: Defined all functional and non-functional requirements, designed system architecture, database schema, and API contracts.	Completed
Phase 2	Core Backend Development and User Authentication Module: Implemented Express.js API server, Prisma schema, PostgreSQL setup, JWT authentication, and user management endpoints.	Completed
Phase 3	File Upload and Presentation Management Modules: Implemented file upload with multer, YouTube URL processing with yt-dlp, FFmpeg audio extraction, and presentation CRUD operations.	Completed
Phase 4	AI Transcription and Content Analysis Integration (AssemblyAI): Implemented AssemblyAI API integration with polling mechanism, transcript storage, and error handling.	Completed
Phase 5	AI-Based Slide Structuring using LSA + YAKE: Implemented Python FastAPI microservice with LSA topic segmentation, YAKE keyword extraction, and structured JSON output.	Completed
Phase 6	Slide Design Enhancement using Google Gemini and PPTX Generation: Integration with Gemini 2.5 for design enhancement and python-pptx for final PPTX file generation.	Completed

Table 4.4: Project Roadmap and Phase Status

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 LIMITATIONS

While the Automated Presentation Generation from Video Using AI system successfully achieves its primary objectives, it has several limitations that should be acknowledged for a complete understanding of the system's current capabilities and boundaries:

- **Dependency on External AI Services:** The system relies on AssemblyAI for transcription and Google Gemini for design enhancement. Both are paid cloud APIs with usage-based pricing. While costs are reasonable for individual use, high-volume institutional deployment could incur significant API costs. Service outages or API changes from these providers could affect system availability and functionality.
- **Audio Quality Sensitivity:** The transcription accuracy of AssemblyAI, while generally high, degrades significantly with poor audio quality, heavy background noise, strong accents, or overlapping speakers. Videos recorded in uncontrolled environments (such as outdoor lectures or informal meetings) may produce transcripts with higher error rates, leading to lower quality slide content.
- **Language Limitation:** The NLP processing pipeline using LSA and YAKE was primarily developed and tested with English-language content. While YAKE supports multiple languages, the stop word lists and preprocessing steps are currently configured for English. Processing videos in other languages may produce suboptimal results.
- **Local File Storage Scalability:** The current implementation stores uploaded files in a local server directory rather than a distributed cloud storage service. This approach limits horizontal scalability, as multiple server instances would not share file access. For production deployment at scale, migration to cloud storage (AWS S3, Google Cloud Storage) is required.
- **Absence of Visual Content Analysis:** The system processes only the audio track of videos, ignoring potentially important visual information including slides being

presented, diagrams, charts, and text displayed on screen. Integration of computer vision for screen capture analysis could significantly improve the relevance and completeness of generated slide content.

- **PPTX Generation Module Completion:** As noted in the deviations section, the PPTX generation module was still in progress at the time of the Second Review. The complete end-to-end pipeline from transcription to downloadable .pptx file was not yet fully functional as of March 2026.
- **No Real-Time Progress Updates:** The current implementation does not provide real-time progress updates to the user during the processing pipeline. Users must wait for the entire pipeline to complete (potentially 2-5 minutes) before receiving their generated presentation. This could negatively impact user experience for longer videos.

5.2 FUTURE ENHANCEMENT

Building on the current implementation, several significant enhancements are proposed for future development to expand the system's capabilities, improve user experience, and address the identified limitations:

- **Visual Content Analysis Integration:** Implementing computer vision using OpenCV or Google Cloud Vision API to extract and analyze video frame content, including screen-captured slides, diagrams, and text on screen. This would enable the system to incorporate visual information into the generated slides, dramatically improving content completeness and relevance for videos that include visual presentations.
- **Real-Time Processing Status:** Implementing WebSocket-based real-time progress updates using Socket.io would allow users to monitor the processing pipeline in real time, seeing updates as transcription completes, NLP processing runs, and slides are generated. This would significantly improve the user experience for longer processing jobs.
- **Local LLM Integration:** Replacing or supplementing the Google Gemini API with locally deployed open-source LLMs (such as Llama 3, Mistral, or Phi-3) would eliminate API costs and data privacy concerns for sensitive content. Tools like Ollama

provide a straightforward API for running LLMs locally, enabling cost-effective deployment at scale.

- **Multi-Language Support:** Extending the NLP pipeline to support multiple languages through multilingual stop word lists, language detection using libraries such as `langdetect`, and multilingual keyword extraction models. This would make the system accessible to a global user base.
- **Cloud Storage Migration:** Migrating file storage from the local file system to AWS S3 or Google Cloud Storage would enable horizontal scaling, improved reliability through geographic redundancy, and reduced storage costs compared to dedicated server storage. The database schema is already designed to support this migration through the `cloudUrl` field in the `File` model.
- **Mobile Application:** Developing a React Native or Flutter mobile application that provides access to the presentation generation functionality on iOS and Android devices, enabling users to submit YouTube URLs or audio recordings directly from their smartphones for automatic slide generation.
- **Advanced Slide Customization:** Implementing a more sophisticated slide editor with drag-and-drop functionality, image insertion, formatting tools, and theme customization would allow users to tailor the generated presentations more precisely to their needs before downloading.
- **Analytics and Insights Dashboard:** Adding a comprehensive analytics dashboard that shows users statistics about their generated presentations, including total slides generated, most common topics across their videos, processing time trends, and transcription accuracy estimates.
- **Collaborative Features:** Implementing WebSocket-based real-time collaborative editing using operational transformation or CRDT (Conflict-free Replicated Data Types) algorithms would enable multiple users to edit the same presentation simultaneously, making the system suitable for team environments.
- **Integration with LMS Platforms:** Developing plugins or API integrations for popular Learning Management Systems including Moodle, Canvas, and Blackboard would enable educational institutions to deploy the system as a native feature within their existing infrastructure.

5.3 CONCLUSION

The project titled "Automated Presentation Generation from Video Using Artificial Intelligence" successfully demonstrates the feasibility and practical value of using a multi-service AI pipeline to automate the conversion of unstructured video and audio content into professionally formatted PowerPoint presentations. By intelligently combining modern AI technologies including cloud-based speech recognition, unsupervised NLP algorithms, and generative AI for design enhancement, the system addresses a real and significant pain point faced by students, educators, and professionals who regularly work with video-based content.

The system's microservice architecture, separating the Node.js Express API gateway from the Python FastAPI NLP processing service, represents an important architectural decision that leverages the respective strengths of each ecosystem. The JavaScript ecosystem provides excellent asynchronous I/O handling and rich API integration capabilities, while the Python ecosystem offers superior NLP libraries and AI tooling. This separation also enables independent scaling and maintenance of each service, which is essential for a production-grade system.

The three-stage AI pipeline — transcription with AssemblyAI, content structuring with LSA and YAKE, and design enhancement with Google Gemini 2.5 — provides a comprehensive transformation from raw multimedia content to polished presentation material. Each stage addresses a distinct challenge in the content transformation process, and their combination produces results that would be difficult or impossible to achieve with any single AI service alone. The LSA+YAKE combination is particularly noteworthy as an efficient, unsupervised approach that does not require training data, making it adaptable to any domain or topic without fine-tuning.

The system's web-based delivery model, with a modern Next.js frontend, responsive design, and intuitive user interface, ensures accessibility for users with varying levels of technical expertise. The presentation management features including the inline slide editor, dashboard analytics, and direct PPTX download provide a complete user experience that supports the full lifecycle of a generated presentation from creation to refinement to use.

From a technical evaluation perspective, the completed modules have demonstrated strong performance across all tested scenarios. The authentication system correctly handles all tested security scenarios. The video upload and YouTube processing modules successfully handle all supported input types with appropriate error handling. The AssemblyAI transcription integration provides high-quality transcripts for clear audio content with robust polling and error handling. The LSA+YAKE NLP pipeline produces semantically coherent slide structures that accurately reflect the content and organization of the source video.

The deviations from the original design specification, while representing changes from the initially proposed approach, are well-justified and result in a higher-quality system overall. The decision to adopt a microservice architecture, use AssemblyAI instead of local Whisper, and leverage Gemini for design enhancement rather than building a custom layout engine each represents a pragmatic engineering trade-off that prioritizes system quality and reliability over strict adherence to the original proposal.

In conclusion, the Automated Presentation Generation from Video Using AI project makes a meaningful contribution to the intersection of AI-powered educational technology and multimedia content management. It demonstrates how the thoughtful combination of cloud AI services, NLP algorithms, and modern full-stack web development can automate a complex, multi-step knowledge transformation process that previously required significant human

time and expertise. The system lays a strong foundation for future enhancements including visual content analysis, multi-language support, mobile applications, and LMS integrations that could further expand its impact and accessibility.

The successful completion of this project as part of the Master of Computer Applications program at Anna University's Centre for Distance Education demonstrates the practical application of advanced computing concepts including machine learning, cloud computing, API integration, database design, and full-stack web development in solving real-world problems. The knowledge and skills acquired through this project provide a strong foundation for future work in AI-powered application development.

REFERENCES

- [1] Park, J., et al. (2022). "A Comprehensive Survey on Automatic Speech Recognition Systems: From HMM to Transformer-based Architectures." *IEEE Transactions on Audio, Speech, and Language Processing*, 30, 45-67.
- [2] Landauer, T. K., and Dumais, S. T. (1997). "A Solution to Plato's Problem: The Latent Semantic Analysis Theory of Acquisition, Induction, and Representation of Knowledge." *Psychological Review*, 104(2), 211-240.
- [3] Steinberger, J., and Jezek, K. (2004). "Using Latent Semantic Analysis in Text Summarization and Summary Evaluation." *Proceedings of ISIM '04*, 93-100.
- [4] Campos, R., et al. (2020). "YAKE! Keyword extraction from single documents using multiple local features." *Information Sciences*, 509, 257-289.
- [5] Sefid, A., et al. (2019). "Automatic Slide Generation from Scientific Papers." *Proceedings of the 3rd Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage*, 80-87.
- [6] Hu, Y., et al. (2015). "PPSGen: Learning-Based Presentation Slides Generation for Academic Papers." *IEEE Transactions on Knowledge and Data Engineering*, 27(4), 1085-1097.
- [7] Kan, M. Y. (2007). "SlideSeer: A Digital Library of Aligned Document and Presentation Pairs." *Proceedings of the 7th ACM/IEEE-CS Joint Conference on Digital Libraries*, 81-90.
- [8] Wei, J., et al. (2022). "Emergent Abilities of Large Language Models." *Transactions on Machine Learning Research*. ISSN 2835-8856.

[9] Shou, Z., et al. (2016). "Temporal Action Localization in Untrimmed Videos via Multi-level Activity Detection." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 3543-3552.

[10] Zhu, Yi., et al. (2020). "A Comprehensive Study of Deep Video Action Recognition." arXiv preprint arXiv:2012.06567.

[11] Newman, S. (2019). Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith. O'Reilly Media. ISBN: 978-1492047841.

[12] Richardson, C., and Smith, F. (2016). Microservices: From Design to Deployment. NGINX, Inc. Available at:
<https://www.nginx.com/resources/library/designing-deploying-microservices/>

[13] AssemblyAI Documentation. "Speech-to-Text and Audio Intelligence API." Available at: <https://www.assemblyai.com/docs> [Accessed: March 2026]

[14] Google AI Documentation. "Google Gemini 2.5 Pro Generative AI API." Available at: <https://ai.google.dev> [Accessed: March 2026]

[15] Prisma Documentation. "Prisma ORM – Database Modelling and Query Engine." Available at: <https://www.prisma.io/docs> [Accessed: March 2026]

APPENDICES

APPENDIX 1: SYSTEM INSTALLATION AND SETUP GUIDE

This appendix provides step-by-step instructions for installing and configuring the Automated Presentation Generation system in a development environment.

Prerequisites

- Node.js 20 LTS or higher (<https://nodejs.org>)
- Python 3.11 or higher (<https://python.org>)
- PostgreSQL 15 or higher (<https://postgresql.org>)
- FFmpeg 6.x (<https://ffmpeg.org>)
- yt-dlp (<https://github.com/yt-dlp/yt-dlp>)
- Git (<https://git-scm.com>)

Backend 1 (Express.js) Setup

- Clone the Express.js backend repository: `git clone [repository-url] backend-express`
 - Navigate to the backend directory: `cd backend-express`
 - Install Node.js dependencies: `npm install`
 - Create the environment configuration file: `cp .env.example .env`
 - Configure the following environment variables in `.env`: `DATABASE_URL` (PostgreSQL connection string), `JWT_SECRET` (random 64-character string), `ASSEMBLYAI_API_KEY` (from AssemblyAI dashboard), `GOOGLE_AI_API_KEY` (from Google AI Studio), `PYTHON_SERVICE_URL` (`http://localhost:8000`)
 - Run Prisma migrations: `npx prisma migrate dev`
 - Start the development server: `npm run dev`
- This appendix provides step-by-step instructions for installing and configuring the Automated Presentation Generation system in a development environment.

Backend 2 (Python FastAPI) Setup

- Clone the Python FastAPI repository: `git clone [repository-url] backend-python`
- Create a Python virtual environment: `python -m venv venv`
- Activate the virtual environment: `source venv/bin/activate` (Linux/Mac) or `venv\Scripts\activate` (Windows)
- Install Python dependencies: `pip install -r requirements.txt`
- Start the FastAPI development server: `uvicorn main:app --reload --port 8000`

Frontend (Next.js) Setup

- Clone the Next.js frontend repository: `git clone [repository-url] frontend-nextjs`
- Navigate to the frontend directory: `cd frontend-nextjs`
- Install Node.js dependencies: `npm install`
- Create the environment configuration file: `cp .env.local.example .env.local`
- Configure `NEXT_PUBLIC_API_URL=http://localhost:8080/api/v1`
- Start the development server: `npm run dev`

APPENDIX 2: API RESPONSE EXAMPLES

This appendix provides sample API request and response examples for the key endpoints of the system.

Sample: POST /api/v1/auth/login

Request Body: `{ "email": "user@example.com", "password": "securePassword123" }`

Success Response (200): `{ "data": { "user": { "id": 1, "email": "user@example.com", "name": "Rahul Singh" }, "token": "eyJhbGc..." }, "message": "Login successful" }`

Error Response (401): `{ "data": null, "message": "Invalid email or password" }`

Sample: POST /api/v1/upload/youtube

Request Body: `{ "url": "https://www.youtube.com/watch?v=dQw4w9WgXcQ" }`

Success Response (200): `{ "data": { "fileId": 42, "title": "Video Title from YouTube", "duration": 213 }, "message": "YouTube video queued for processing" }`

Sample: GET /api/v1/presentations/5/slides

Success Response (200): `{ "data": { "presentationId": 5, "slides": [{ "slideIndex": 0, "title": "Introduction to Machine Learning", "bulletPoints": ["Definition and core concepts", "Types: supervised, unsupervised, reinforcement", "Real-world applications"], "speakerNotes": "Begin with a broad overview..." }] }, "message": "Slides retrieved successfully" }`

APPENDIX 3: NLP ALGORITHM IMPLEMENTATION DETAILS

This appendix provides a detailed description of the LSA and YAKE algorithms as implemented in the Python FastAPI microservice.

LSA Implementation Steps

- **Text Preprocessing:** The raw transcript is tokenized using NLTK's word tokenizer. Stop words from NLTK's English stop word corpus are removed. Remaining tokens are stemmed using the Porter Stemmer for dimensionality reduction.
- **Sentence Matrix Construction:** The transcript is divided into individual sentences using NLTK's `sent_tokenize` function. A TF-IDF matrix is computed with each sentence as a document, using scikit-learn's `TfidfVectorizer` with `max_features=5000` to limit vocabulary size.
- **Singular Value Decomposition:** `TruncatedSVD` from scikit-learn is applied to the TF-IDF matrix, projecting sentences into a lower-dimensional semantic space. The number of components is set to $\min(n_topics, n_sentences - 1)$ where `n_topics` is determined as $\max(3, \text{transcript_word_count} // 200)$.
- **K-Means Clustering:** K-means clustering is applied to the SVD-transformed sentence matrix with `k` equal to the target number of slides. Cluster centroids are identified as the sentences with the minimum distance to each cluster centroid.
- **Slide Content Assembly:** For each cluster, all sentences are sorted by their cluster membership score. The top 3-5 sentences form the source content for each slide. YAKE processes each cluster's text independently to extract the most important keyphrases as bullet points.